

English(original) 中文

Commit at: 31 Aug 2025 New commit available, click to sync

Qwen Code Configuration

Qwen Code offers several ways to configure its behavior, including environment variables, command-line arguments, and settings files. This document outlines the different configuration methods and available settings.

Configuration layers

Configuration is applied in the following order of precedence (lower numbers are overridden by higher numbers):

Default values: Hardcoded defaults within the application.

User settings file: Global settings for the current user.

Project settings file: Project-specific settings.

System settings file: System-wide settings.

Environment variables: System-wide or session-specific variables, potentially loaded from `.env` files.

Command-line arguments: Values passed when launching the CLI.

Settings files

Qwen Code uses `settings.json` files for persistent configuration. There are three locations for these files:

User settings file:

Location: `~/.qwen/settings.json` (where `~` is your home directory).

Scope: Applies to all Qwen Code sessions for the current user.

Project settings file:

Location: `.qwen/settings.json` within your project's root directory.

Scope: Applies only when running Qwen Code from that specific project. Project settings override user settings.

System settings file:

Location: `/etc/gemini-cli/settings.json` (Linux), `C:\ProgramData\gemini-cli\settings.json` (Windows) or `/Library/Application Support/GeminiCli/settings.json` (macOS). The path can be overridden using the `GEMINI_CLI_SYSTEM_SETTINGS_PATH` environment variable.

Scope: Applies to all Qwen Code sessions on the system, for all users. System settings override user and project settings. May be useful for system administrators at enterprises to have controls over users' Qwen Code setups.

Note on environment variables in settings: String values within your `settings.json` files can reference environment variables using either `$VAR_NAME` or `${VAR_NAME}` syntax. These variables will be automatically resolved when the settings are loaded. For example, if you have an environment variable `MY_API_TOKEN`, you could use it in `settings.json` like this: `"apiKey": "$MY_API_TOKEN"`.

The `.qwen` directory in your project

In addition to a project settings file, a project's `.qwen` directory can contain other project-specific files related to Qwen Code's operation, such as:

[Custom sandbox profiles](#) (e.g., `.qwen/sandbox-macos-custom.sb`, `.qwen/sandbox.Dockerfile`).

Available settings in `settings.json`:

contextFileName (string or array of strings):

Description: Specifies the filename for context files (e.g., `QWEN.md`, `AGENTS.md`). Can be a single filename or a list of accepted filenames.

Default: `QWEN.md`

Example: `"contextFileName": "AGENTS.md"`

bugCommand (object):

Description: Overrides the default URL for the `/bug` command.

Default: `"urlTemplate": "https://github.com/QwenLM/qwen-code/issues/new?template=bug_report.yml&title={title}&info={info}"`

Properties:

urlTemplate (string): A URL that can contain `{title}` and `{info}` placeholders.

Example:

```
"bugCommand": {
  "urlTemplate": "https://bug.example.com/new?title={title}&info={info}"
}
```

```
}
```

fileFiltering (object):

Description: Controls git-aware file filtering behavior for @ commands and file discovery tools.

Default: "respectGitIgnore": true, "enableRecursiveFileSearch": true

Properties:

respectGitIgnore (boolean): Whether to respect .gitignore patterns when discovering files. When set to true, git-ignored files (like node_modules/, dist/, .env) are automatically excluded from @ commands and file listing operations.

enableRecursiveFileSearch (boolean): Whether to enable searching recursively for filenames under the current tree when completing @ prefixes in the prompt.

Example:

```
"fileFiltering": {
  "respectGitIgnore": true,
  "enableRecursiveFileSearch": false
}
```

coreTools (array of strings):

Description: Allows you to specify a list of core tool names that should be made available to the model. This can be used to restrict the set of built-in tools. See [Built-in Tools](#) for a list of core tools. You can also specify command-specific restrictions for tools that support it, like the ShellTool. For example, "coreTools": ["ShellTool(ls -l)"] will only allow the ls -l command to be executed.

Default: All tools available for use by the model.

Example: "coreTools": ["ReadFileTool", "GlobTool", "ShellTool(ls)"] .

excludeTools (array of strings):

Description: Allows you to specify a list of core tool names that should be excluded from the model. A tool listed in both excludeTools and coreTools is excluded. You can also specify command-specific restrictions for tools that support it, like the ShellTool. For example, "excludeTools": ["ShellTool(rm -rf)"] will block the rm -rf command.

Default: No tools excluded.

Example: "excludeTools": ["run_shell_command", "findFiles"] .

Security Note: Command-specific restrictions in `excludeTools` for `run_shell_command` are based on simple string matching and can be easily bypassed. This feature is **not a security mechanism** and should not be relied upon to safely execute untrusted code. It is recommended to use `coreTools` to explicitly select commands that can be executed.

allowMCPServers (array of strings):

Description: Allows you to specify a list of MCP server names that should be made available to the model. This can be used to restrict the set of MCP servers to connect to. Note that this will be ignored if `--allowed-mcp-server-names` is set.

Default: All MCP servers are available for use by the model.

Example: `"allowMCPServers": ["myPythonServer"]` .

Security Note: This uses simple string matching on MCP server names, which can be modified. If you're a system administrator looking to prevent users from bypassing this, consider configuring the `mcpServers` at the system settings level such that the user will not be able to configure any MCP servers of their own. This should not be used as an airtight security mechanism.

excludeMCPServers (array of strings):

Description: Allows you to specify a list of MCP server names that should be excluded from the model. A server listed in both `excludeMCPServers` and `allowMCPServers` is excluded. Note that this will be ignored if `--allowed-mcp-server-names` is set.

Default: No MCP servers excluded.

Example: `"excludeMCPServers": ["myNodeServer"]` .

Security Note: This uses simple string matching on MCP server names, which can be modified. If you're a system administrator looking to prevent users from bypassing this, consider configuring the `mcpServers` at the system settings level such that the user will not be able to configure any MCP servers of their own. This should not be used as an airtight security mechanism.

autoAccept (boolean):

Description: Controls whether the CLI automatically accepts and executes tool calls that are considered safe (e.g., read-only operations) without explicit user confirmation. If set to `true` , the CLI will bypass the confirmation prompt for tools deemed safe.

Default: `false`

Example: `"autoAccept": true`

theme (string):

Description: Sets the visual [theme](#) for Qwen Code.

Default: "Default"

Example: "theme": "GitHub"

vimMode (boolean):

Description: Enables or disables vim mode for input editing. When enabled, the input area supports vim-style navigation and editing commands with NORMAL and INSERT modes. The vim mode status is displayed in the footer and persists between sessions.

Default: false

Example: "vimMode": true

sandbox (boolean or string):

Description: Controls whether and how to use sandboxing for tool execution. If set to true, Qwen Code uses a pre-built `qwen-code-sandbox` Docker image. For more information, see [Sandboxing](#).

Default: false

Example: "sandbox": "docker"

toolDiscoveryCommand (string):

Description: Defines a custom shell command for discovering tools from your project. The shell command must return on `stdout` a JSON array of [function declarations](#). Tool wrappers are optional.

Default: Empty

Example: "toolDiscoveryCommand": "bin/get_tools"

toolCallCommand (string):

Description: Defines a custom shell command for calling a specific tool that was discovered using `toolDiscoveryCommand`. The shell command must meet the following criteria:

- It must take `function name` (exactly as in [function declaration](#)) as first command line argument.

- It must read function arguments as JSON on `stdin`, analogous to [functionCall.args](#).

- It must return function output as JSON on `stdout`, analogous to [functionResponse.response.content](#).

Default: Empty

Example: "toolCallCommand": "bin/call_tool"

mcpServers (object):

Description: Configures connections to one or more Model-Context Protocol (MCP) servers for discovering and using custom tools. Qwen Code attempts to connect to each configured MCP server to discover available tools. If multiple MCP servers expose a tool with the same name, the tool names will be prefixed with the server alias you defined in the configuration (e.g., `serverAlias__actualToolName`) to avoid conflicts. Note that the system might strip certain schema properties from MCP tool definitions for compatibility. At least one of `command`, `url`, or `httpUrl` must be provided. If multiple are specified, the order of precedence is `httpUrl`, then `url`, then `command`.

Default: Empty

Properties:

<SERVER_NAME> (object): The server parameters for the named server.

`command` (string, optional): The command to execute to start the MCP server via standard I/O.

`args` (array of strings, optional): Arguments to pass to the command.

`env` (object, optional): Environment variables to set for the server process.

`cwd` (string, optional): The working directory in which to start the server.

`url` (string, optional): The URL of an MCP server that uses Server-Sent Events (SSE) for communication.

`httpUrl` (string, optional): The URL of an MCP server that uses streamable HTTP for communication.

`headers` (object, optional): A map of HTTP headers to send with requests to `url` or `httpUrl`.

`timeout` (number, optional): Timeout in milliseconds for requests to this MCP server.

`trust` (boolean, optional): Trust this server and bypass all tool call confirmations.

`description` (string, optional): A brief description of the server, which may be used for display purposes.

`includeTools` (array of strings, optional): List of tool names to include from this MCP server. When specified, only the tools listed here will be available from this server (whitelist behavior). If not specified, all tools from the server are enabled by default.

`excludeTools` (array of strings, optional): List of tool names to exclude from this MCP server. Tools listed here will not be available to the model, even if they are exposed by the server. **Note:** `excludeTools` takes precedence over `includeTools` - if a tool is in both lists, it will be excluded.

Example:

```

"mcpServers": {
  "myPythonServer": {
    "command": "python",
    "args": ["mcp_server.py", "--port", "8080"],
    "cwd": "./mcp_tools/python",
    "timeout": 5000,
    "includeTools": ["safe_tool", "file_reader"],
  },
  "myNodeServer": {
    "command": "node",
    "args": ["mcp_server.js"],
    "cwd": "./mcp_tools/node",
    "excludeTools": ["dangerous_tool", "file_deleter"]
  },
  "myDockerServer": {
    "command": "docker",
    "args": ["run", "-i", "--rm", "-e", "API_KEY", "ghcr.io/foo/bar"],
    "env": {
      "API_KEY": "$MY_API_TOKEN"
    }
  },
  "mySseServer": {
    "url": "http://localhost:8081/events",
    "headers": {
      "Authorization": "Bearer $MY_SSE_TOKEN"
    },
    "description": "An example SSE-based MCP server."
  },
  "myStreamableHttpServer": {
    "httpUrl": "http://localhost:8082/stream",
    "headers": {
      "X-API-Key": "$MY_HTTP_API_KEY"
    },
    "description": "An example Streamable HTTP-based MCP server."
  }
}

```

checkpointing (object):

Description: Configures the checkpointing feature, which allows you to save and restore conversation and file states. See the [Checkpointing documentation](#) for more details.

Default: {"enabled": false}

Properties:

enabled (boolean): When `true`, the `/restore` command is available.

preferredEditor (string):

Description: Specifies the preferred editor to use for viewing diffs.

Default: `vscode`

Example: `"preferredEditor": "vscode"`

telemetry (object)

Description: Configures logging and metrics collection for Qwen Code. For more information, see [Telemetry](#).

Default: `{"enabled": false, "target": "local", "otlpEndpoint": "http://localhost:4317", "logPrompts": true}`

Properties:

enabled (boolean): Whether or not telemetry is enabled.

target (string): The destination for collected telemetry. Supported values are `local` and `gcp`.

otlpEndpoint (string): The endpoint for the OTLP Exporter.

logPrompts (boolean): Whether or not to include the content of user prompts in the logs.

Example:

```
"telemetry": {  
  "enabled": true,  
  "target": "local",  
  "otlpEndpoint": "http://localhost:16686",  
  "logPrompts": false  
}
```

usageStatisticsEnabled (boolean):

Description: Enables or disables the collection of usage statistics. See [Usage Statistics](#) for more information.

Default: `true`

Example:

```
"usageStatisticsEnabled": false
```


hideTips (boolean):

Description: Enables or disables helpful tips in the CLI interface.

Default: false

Example:

```
"hideTips": true
```

hideBanner (boolean):

Description: Enables or disables the startup banner (ASCII art logo) in the CLI interface.

Default: false

Example:

```
"hideBanner": true
```

maxSessionTurns (number):

Description: Sets the maximum number of turns for a session. If the session exceeds this limit, the CLI will stop processing and start a new chat.

Default: -1 (unlimited)

Example:

```
"maxSessionTurns": 10
```

summarizeToolOutput (object):

Description: Enables or disables the summarization of tool output. You can specify the token budget for the summarization using the `tokenBudget` setting.

Note: Currently only the `run_shell_command` tool is supported.

Default: {} (Disabled by default)

Example:

```

"summarizeToolOutput": {
  "run_shell_command": {
    "tokenBudget": 2000
  }
}

```

excludedProjectEnvVars (array of strings):

Description: Specifies environment variables that should be excluded from being loaded from project `.env` files. This prevents project-specific environment variables (like `DEBUG=true`) from interfering with the CLI behavior. Variables from `.qwen/.env` files are never excluded.

Default: `["DEBUG", "DEBUG_MODE"]`

Example:

```

"excludedProjectEnvVars": ["DEBUG", "DEBUG_MODE", "NODE_ENV"]

```

includeDirectories (array of strings):

Description: Specifies an array of additional absolute or relative paths to include in the workspace context. This allows you to work with files across multiple directories as if they were one. Paths can use `~` to refer to the user's home directory. This setting can be combined with the `--include-directories` command-line flag.

Default: `[]`

Example:

```

"includeDirectories": [
  "/path/to/another/project",
  "../shared-library",
  "~/common-utils"
]

```

loadMemoryFromIncludeDirectories (boolean):

Description: Controls the behavior of the `/memory refresh` command. If set to `true`, `QWEN.md` files should be loaded from all directories that are added. If set to `false`, `QWEN.md` should only be loaded from the current directory.

Default: `false`

Example:

```
"loadMemoryFromIncludeDirectories": true
```

tavilyApiKey (string):

Description: API key for Tavily web search service. Required to enable the `web_search` tool functionality. If not configured, the web search tool will be disabled and skipped.

Default: undefined (web search disabled)

Example: "tavilyApiKey": "tvly-your-api-key-here"

chatCompression (object):

Description: Controls the settings for chat history compression, both automatic and when manually invoked through the `/compress` command.

Properties:

contextPercentageThreshold (number): A value between 0 and 1 that specifies the token threshold for compression as a percentage of the model's total token limit. For example, a value of `0.6` will trigger compression when the chat history exceeds 60% of the token limit.

Example:

```
"chatCompression": {  
  "contextPercentageThreshold": 0.6  
}
```

showLineNumbers (boolean):

Description: Controls whether line numbers are displayed in code blocks in the CLI output.

Default: true

Example:

```
"showLineNumbers": false
```

Example settings.json:

```

{
  "theme": "GitHub",
  "sandbox": "docker",
  "toolDiscoveryCommand": "bin/get_tools",
  "toolCallCommand": "bin/call_tool",
  "tavilyApiKey": "$TAVILY_API_KEY",
  "mcpServers": {
    "mainServer": {
      "command": "bin/mcp_server.py"
    },
    "anotherServer": {
      "command": "node",
      "args": ["mcp_server.js", "--verbose"]
    }
  },
  "telemetry": {
    "enabled": true,
    "target": "local",
    "otlpEndpoint": "http://localhost:4317",
    "logPrompts": true
  },
  "usageStatisticsEnabled": true,
  "hideTips": false,
  "hideBanner": false,
  "maxSessionTurns": 10,
  "summarizeToolOutput": {
    "run_shell_command": {
      "tokenBudget": 100
    }
  },
  "excludedProjectEnvVars": ["DEBUG", "DEBUG_MODE", "NODE_ENV"],
  "includeDirectories": ["path/to/dir1", "~/path/to/dir2", "../path/to/dir3"],
  "loadMemoryFromIncludeDirectories": true
}

```

Shell History

The CLI keeps a history of shell commands you run. To avoid conflicts between different projects, this history is stored in a project-specific directory within your user's home folder.

Location: `~/.qwen/tmp/<project_hash>/shell_history`

`<project_hash>` is a unique identifier generated from your project's root path.

The history is stored in a file named `shell_history`.

Environment Variables & .env Files

Environment variables are a common way to configure applications, especially for sensitive information like API keys or for settings that might change between environments. For authentication setup, see the [Authentication documentation](#) which covers all available authentication methods.

The CLI automatically loads environment variables from an `.env` file. The loading order is:

- `.env` file in the current working directory.

- If not found, it searches upwards in parent directories until it finds an `.env` file or reaches the project root (identified by a `.git` folder) or the home directory.

- If still not found, it looks for `~/.env` (in the user's home directory).

Environment Variable Exclusion: Some environment variables (like `DEBUG` and `DEBUG_MODE`) are automatically excluded from project `.env` files by default to prevent interference with the CLI behavior. Variables from `.qwen/.env` files are never excluded. You can customize this behavior using the `excludedProjectEnvVars` setting in your `settings.json` file.

GEMINI_API_KEY :

- Your API key for the Gemini API.

- One of several available [authentication methods](#).

- Set this in your shell profile (e.g., `~/.bashrc` , `~/.zshrc`) or an `.env` file.

GEMINI_MODEL :

- Specifies the default Gemini model to use.

- Overrides the hardcoded default

- Example: `export GEMINI_MODEL="gemini-2.5-flash"`

GOOGLE_API_KEY :

- Your Google Cloud API key.

- Required for using Vertex AI in express mode.

- Ensure you have the necessary permissions.

- Example: `export GOOGLE_API_KEY="YOUR_GOOGLE_API_KEY"` .

GOOGLE_CLOUD_PROJECT :

- Your Google Cloud Project ID.

- Required for using Code Assist or Vertex AI.

- If using Vertex AI, ensure you have the necessary permissions in this project.

Cloud Shell Note: When running in a Cloud Shell environment, this variable defaults to a special project allocated for Cloud Shell users. If you have `GOOGLE_CLOUD_PROJECT` set

in your global environment in Cloud Shell, it will be overridden by this default. To use a different project in Cloud Shell, you must define `GOOGLE_CLOUD_PROJECT` in a `.env` file.

Example: `export GOOGLE_CLOUD_PROJECT="YOUR_PROJECT_ID"` .

GOOGLE_APPLICATION_CREDENTIALS (string):

Description: The path to your Google Application Credentials JSON file.

Example: `export GOOGLE_APPLICATION_CREDENTIALS="/path/to/your/credentials.json"`

OTLP_GOOGLE_CLOUD_PROJECT :

Your Google Cloud Project ID for Telemetry in Google Cloud

Example: `export OTLP_GOOGLE_CLOUD_PROJECT="YOUR_PROJECT_ID"` .

GOOGLE_CLOUD_LOCATION :

Your Google Cloud Project Location (e.g., us-central1).

Required for using Vertex AI in non express mode.

Example: `export GOOGLE_CLOUD_LOCATION="YOUR_PROJECT_LOCATION"` .

GEMINI_SANDBOX :

Alternative to the `sandbox` setting in `settings.json` .

Accepts `true` , `false` , `docker` , `podman` , or a custom command string.

SEATBELT_PROFILE (macOS specific):

zdoc [QwenLM/qwen-code](#)  **Github Trending**

English 



other operations.

`strict` : Uses a strict profile that declines operations by default.

`<profile_name>` : Uses a custom profile. To define a custom profile, create a file named `sandbox-macos-<profile_name>.sb` in your project's `.qwen/` directory (e.g., `my-project/.qwen/sandbox-macos-custom.sb`).

DEBUG or **DEBUG_MODE** (often used by underlying libraries or the CLI itself):

Set to `true` or `1` to enable verbose debug logging, which can be helpful for troubleshooting.

Note: These variables are automatically excluded from project `.env` files by default to prevent interference with the CLI behavior. Use `.qwen/.env` files if you need to set these for Qwen Code specifically.

NO_COLOR :

Set to any value to disable all color output in the CLI.

CLI_TITLE :

Set to a string to customize the title of the CLI.

CODE_ASSIST_ENDPOINT :

Specifies the endpoint for the code assist server.

This is useful for development and testing.

TAVILY_API_KEY :

Your API key for the Tavily web search service.

Required to enable the `web_search` tool functionality.

If not configured, the web search tool will be disabled and skipped.

Example: `export TAVILY_API_KEY="tvly-your-api-key-here"`

Command-Line Arguments

Arguments passed directly when running the CLI can override other configurations for that specific session.

--model <model_name> (-m <model_name>):

Specifies the Gemini model to use for this session.

Example: `npm start -- --model gemini-1.5-pro-latest`

--prompt <your_prompt> (-p <your_prompt>):

Used to pass a prompt directly to the command. This invokes Qwen Code in a non-interactive mode.

--prompt-interactive <your_prompt> (-i <your_prompt>):

Starts an interactive session with the provided prompt as the initial input.

The prompt is processed within the interactive session, not before it.

Cannot be used when piping input from stdin.

Example: `qwen -i "explain this code"`

--sandbox (-s):

Enables sandbox mode for this session.

--sandbox-image :

Sets the sandbox image URI.

--debug (-d):

Enables debug mode for this session, providing more verbose output.

--all-files (-a):

If set, recursively includes all files within the current directory as context for the prompt.

--help (or -h):

Displays help information about command-line arguments.

--show-memory-usage :

Displays the current memory usage.

--yolo :

Enables YOLO mode, which automatically **approves** all tool calls.

--approval-mode <mode> :

Sets the approval mode for tool calls. Available modes:

default : Prompt for approval on each tool call (default behavior)

auto_edit : Automatically **approve** edit tools (replace, write_file) while prompting for others

yo!o : Automatically **approve** all tool calls (equivalent to --yo!o)

Cannot be used together with --yo!o . Use --approval-mode=yo!o instead of --yo!o for the new unified approach.

Example: qwen --approval-mode auto_edit

--telemetry :

Enables [telemetry](#).

--telemetry-target :

Sets the telemetry target. See [telemetry](#) for more information.

--telemetry-otlp-endpoint :

Sets the OTLP endpoint for telemetry. See [telemetry](#) for more information.

--telemetry-otlp-protocol :

Sets the OTLP protocol for telemetry (grpc or http). Defaults to grpc . See [telemetry](#) for more information.

--telemetry-log-prompts :

Enables logging of prompts for telemetry. See [telemetry](#) for more information.

--checkpointing :

Enables [checkpointing](#).

--extensions <extension_name ...> (-e <extension_name ...>):

Specifies a list of extensions to use for the session. If not provided, all available extensions are used.

Use the special term qwen -e none to disable all extensions.

Example: qwen -e my-extension -e my-other-extension

--list-extensions (-l):

Lists all available extensions and exits.

--proxy :

Sets the proxy for the CLI.

Example: --proxy http://localhost:7890 .

--include-directories <dir1,dir2,...> :

Includes additional directories in the workspace for multi-directory support.

Can be specified multiple times or as comma-separated values.

5 directories can be added at maximum.

Example: `--include-directories /path/to/project1,/path/to/project2` Or `--include-directories /path/to/project1 --include-directories /path/to/project2`

--version :

Displays the version of the CLI.

--openai-logging :

Enables logging of OpenAI API calls for debugging and analysis. This flag overrides the `enableOpenAILogging` setting in `settings.json`.

--tavily-api-key <api_key> :

Sets the Tavily API key for web search functionality for this session.

Example: `qwen --tavily-api-key tvly-your-api-key-here`

Context Files (Hierarchical Instructional Context)

While not strictly configuration for the CLI's *behavior*, context files (defaulting to `QWEN.md` but configurable via the `contextFileName` setting) are crucial for configuring the *instructional context* (also referred to as "memory"). This powerful feature allows you to give project-specific instructions, coding style guides, or any relevant background information to the AI, making its responses more tailored and accurate to your needs. The CLI includes UI elements, such as an indicator in the footer showing the number of loaded context files, to keep you informed about the active context.

Purpose: These Markdown files contain instructions, guidelines, or context that you want the Gemini model to be aware of during your interactions. The system is designed to manage this instructional context hierarchically.

Example Context File Content (e.g., `QWEN.md`)

Here's a conceptual example of what a context file at the root of a TypeScript project might contain:

```
# Project: My Awesome TypeScript Library
```

```
## General Instructions:
```

- When generating new TypeScript code, please follow the existing coding style.
- Ensure all new functions and classes have JSDoc comments.
- Prefer functional programming paradigms where appropriate.
- All code should be compatible with TypeScript 5.0 and Node.js 20+.

```
## Coding Style:
```

- Use 2 spaces for indentation.
- Interface names should be prefixed with ``I`` (e.g., ``IUserService``).
- Private class members should be prefixed with an underscore (``_``).
- Always use strict equality (``===`` and ``!==``).

Specific Component: ``src/api/client.ts``

- This file handles all outbound API requests.
- When adding new API call functions, ensure they include robust error handling and logging.
- Use the existing ``fetchWithRetry`` utility for all GET requests.

Regarding Dependencies:

- Avoid introducing new external dependencies unless absolutely necessary.
- If a new dependency is required, please state the reason.

This example demonstrates how you can provide general project context, specific coding conventions, and even notes about particular files or components. The more relevant and precise your context files are, the better the AI can assist you. Project-specific context files are highly encouraged to establish conventions and context.

Hierarchical Loading and Precedence: The CLI implements a sophisticated hierarchical memory system by loading context files (e.g., `QWEN.md`) from several locations. Content from files lower in this list (more specific) typically overrides or supplements content from files higher up (more general). The exact concatenation order and final context can be inspected using the `/memory show` command. The typical loading order is:

i. Global Context File:

Location: `~/.qwen/<contextFileName>` (e.g., `~/.qwen/QWEN.md` in your user home directory).

Scope: Provides default instructions for all your projects.

ii. Project Root & Ancestors Context Files:

Location: The CLI searches for the configured context file in the current working directory and then in each parent directory up to either the project root (identified by a `.git` folder) or your home directory.

Scope: Provides context relevant to the entire project or a significant portion of it.

iii. Sub-directory Context Files (Contextual/Local):

Location: The CLI also scans for the configured context file in subdirectories *below* the current working directory (respecting common ignore patterns like `node_modules`, `.git`, etc.). The breadth of this search is limited to 200 directories by default, but can be configured with a `memoryDiscoveryMaxDirs` field in your `settings.json` file.

Scope: Allows for highly specific instructions relevant to a particular component, module, or subsection of your project.

Concatenation & UI Indication: The contents of all found context files are concatenated (with separators indicating their origin and path) and provided as part of the system prompt. The CLI footer displays the count of loaded context files, giving you a quick visual cue about the active instructional context.

Importing Content: You can modularize your context files by importing other Markdown files using the `@path/to/file.md` syntax. For more details, see the [Memory Import Processor documentation](#).

Commands for Memory Management:

Use `/memory refresh` to force a re-scan and reload of all context files from all configured locations. This updates the AI's instructional context.

Use `/memory show` to display the combined instructional context currently loaded, allowing you to verify the hierarchy and content being used by the AI.

See the [Commands documentation](#) for full details on the `/memory` command and its sub-commands (`show` and `refresh`).

By understanding and utilizing these configuration layers and the hierarchical nature of context files, you can effectively manage the AI's memory and tailor Qwen Code's responses to your specific needs and projects.

Sandboxing

Qwen Code can execute potentially unsafe operations (like shell commands and file modifications) within a sandboxed environment to protect your system.

Sandboxing is disabled by default, but you can enable it in a few ways:

Using `--sandbox` or `-s` flag.

Setting `GEMINI_SANDBOX` environment variable.

Sandbox is enabled when using `--yolo` or `--approval-mode=yolo` by default.

By default, it uses a pre-built `qwen-code-sandbox` Docker image.

For project-specific sandboxing needs, you can create a custom Dockerfile at `.qwen/sandbox.Dockerfile` in your project's root directory. This Dockerfile can be based on the base sandbox image:

```
FROM qwen-code-sandbox
```

```
# Add your custom dependencies or configurations here
# For example:
# RUN apt-get update && apt-get install -y some-package
# COPY ./my-config /app/my-config
```

When `.qwen/sandbox.Dockerfile` exists, you can use `BUILD_SANDBOX` environment variable when running Qwen Code to automatically build the custom sandbox image:

```
BUILD_SANDBOX=1 qwen -s
```

Usage Statistics

To help us improve Qwen Code, we collect anonymized usage statistics. This data helps us understand how the CLI is used, identify common issues, and prioritize new features.

What we collect:

Tool Calls: We log the names of the tools that are called, whether they succeed or fail, and how long they take to execute. We do not collect the arguments passed to the tools or any data returned by them.

API Requests: We log the model used for each request, the duration of the request, and whether it was successful. We do not collect the content of the prompts or responses.

Session Information: We collect information about the configuration of the CLI, such as the enabled tools and the approval mode.

What we DON'T collect:

Personally Identifiable Information (PII): We do not collect any personal information, such as your name, email address, or API keys.

Prompt and Response Content: We do not log the content of your prompts or the responses from the model.

File Content: We do not log the content of any files that are read or written by the CLI.

How to opt out:

You can opt out of usage statistics collection at any time by setting the `usageStatisticsEnabled` property to `false` in your `settings.json` file:

```
{  
  "usageStatisticsEnabled": false  
}
```

Note: When usage statistics are enabled, events are sent to an Alibaba Cloud RUM collection endpoint.

Content sourced from [QwenLM/qwen-code](#), translated by [zdoc](#).